

halfedge：核心拓扑操作模块设计文档

src/topology_ops.rs

halfedge 库

2026-07-04

目录

1	模块定位	3
2	拓扑约定回顾	3
3	错误类型	3
3.1	错误信息语言约定	4
4	validate_mesh：流形不变量校验	4
5	split_edge：边分裂	4
5.1	算法描述	4
5.1.1	内部边（两侧均有面）	5
5.1.2	边界边（仅一侧有面）	5
5.2	数量变化	5
6	flip_edge：内部边翻转	6
6.1	算法描述	6
6.2	重连规则	6
6.3	校验	6
7	collapse_edge：边折叠	7
7.1	算法描述	7
7.2	链接条件	7
7.3	操作步骤	8
7.4	数量变化	8
8	extrude_face：面挤出	8
8.1	算法描述	8
8.1.1	限制条件	8
8.1.2	退化检查	8
8.1.3	创建元素	9

8.1.4	顶面 F'	9
8.1.5	侧面三角形	9
8.1.6	twin 对应 (12 对)	9
8.1.7	顶点 outgoing 入口	10
8.2	校验	10
8.3	数量变化	10
8.4	批量版本	10
9	extrude_region: 区域挤出	10
9.1	顶点分类	10
9.2	半边分类	11
9.3	侧面创建	11
9.4	面环重建	11
9.5	退化检查	11
9.6	边界情况	11
10	算法流程图	12
10.1	split_edge 流程	12
10.2	flip_edge 流程	13
10.3	collapse_edge 流程	14
10.4	extrude_face 流程	15
10.5	extrude_region 流程	16
11	add_triangle: 增量三角形构建	16
11.1	算法步骤	16
11.2	twin 查找策略	17
11.3	数量变化	17
11.4	校验	17
12	split_face: 面分裂 (poke)	17
12.1	算法步骤	18
12.2	拓扑示意	18
12.3	twin 恢复策略	18
12.4	数量变化	18
12.5	注意事项	19
13	复杂度	19
14	单元测试覆盖	19
15	设计取舍	21

1 模块定位

`topology_ops.rs` 在 `storage.rs` (存储) 与 `traversal.rs` (遍历) 之上叠加写操作能力, 提供边/面级拓扑操作、面挤出与增量构建, 每个操作内置合法性校验, 保证操作后网格仍为流形三角曲面。

操作	API	语义
边分裂	<code>split_edge(mesh, he)</code>	在边中点插入新顶点, 1 边 $\rightarrow$ 2 边
边翻转	<code>flip_edge(mesh, he)</code>	替换四边形对角线
边折叠	<code>collapse_edge(mesh, he)</code>	合并两端顶点 (中点), 删除 2 面
	<code>collapse_edge_at(mesh, he, pos)</code>	合并两端顶点 (指定位置), 删除 2 面
面挤出	<code>extrude_face(mesh, f, offset)</code>	三角面沿方向挤出形成棱柱体
批量挤出	<code>extrude_faces(mesh, &[f], offset)</code>	多个面同时挤出
区域挤出	<code>extrude_region(mesh, &[f], offset)</code>	多个相邻面区域挤出, 含内部/边界顶点分类
面分裂	<code>split_face(mesh, f)</code>	面内插入重心顶点, 1 三角 $\rightarrow$ 3 三角 (poke)
增量构建	<code>add_triangle(mesh, v0, v1, v2)</code>	向网格追加一个三角形, 自动缝 twin
校验	<code>validate_mesh(mesh)</code>	检查流形三角曲面不变量

2 拓扑约定回顾

- `HalfEdge.vertex` 是 tip (目的顶点), `origin = twin.vertex`;
- 同面 `next/prev` 形成 CCW 闭合环;
- `twin` 互指, 构成无向边;
- 边界半边 `face = None`。

设 h 是一条半边, $A = h.twin.vertex$ (origin), $B = h.vertex$ (tip), 则 h 表示有向边 $A \rightarrow B$ 。

3 错误类型

所有操作返回 `Result<T, TopologyError>`, 失败时不修改网格 (原子性)。

变体	触发场景
<code>InvalidHalfEdge(he)</code>	句柄无效或已删除
<code>FlipOnBoundaryEdge(he)</code>	试图翻转边界边
<code>CollapseOnBoundaryEdge(he)</code>	试图折叠边界边
<code>NoTwin(he)</code>	半边没有 twin
<code>NoFace(he)</code>	两侧均无面
<code>DegenerateTriangle</code>	$C = D$ 、offset 零向量或侧面退化
<code>FlipCreatesNonManifoldEdge{c, d}</code>	翻转后对角线两端已相连（产生重复边）
<code>LinkConditionViolated{a, b}</code>	公共邻居 $\neq \{C, D\}$
<code>CollapseOnBoundaryVertices{a, b}</code>	折叠边两端点均为边界顶点
<code>Inconsistent(msg)</code>	twin 不互指、next/prev 断裂、面含内部边等

3.1 错误信息语言约定

`TopologyError::Inconsistent(msg)` 中的 `msg` 字符串**统一使用中文**,与代码库其余部分(`io.rs` 各 `*Error`、`validate.rs`、`panic!` / `eprintln!` / `expect()`)保持一致。例如:

- 字段缺失类: `"h.next      None", "twin.prev      None", "p1.twin      None"`;
- 实体缺失类: `"          A                  ", "          {?:?}                  "`;
- 几何/语义类: `"                          ", "                          ", "                          {?:?}                          ={:},                          "`。

半边字段路径保留原文: `h.next / h.prev / twin.next / twin.prev / p1.twin / n1.twin / n2.twin / p2.twin` 等以字段名直接引用结构体成员,便于从错误信息定位到对应源码字段。

4 validate_mesh: 流形不变量校验

`validate_mesh(mesh)` 遍历所有半边与面,检查以下不变量:

1. **twin 互指**: $h.twin = t \Rightarrow t.twin = h$;
2. **无自环**: $h.twin.vertex \neq h.vertex$;
3. **next/prev 一致**: $h.next = n \Rightarrow n.prev = h$;
4. **三角面**: 每个面的边界环长度 = 3。

5 split_edge: 边分裂

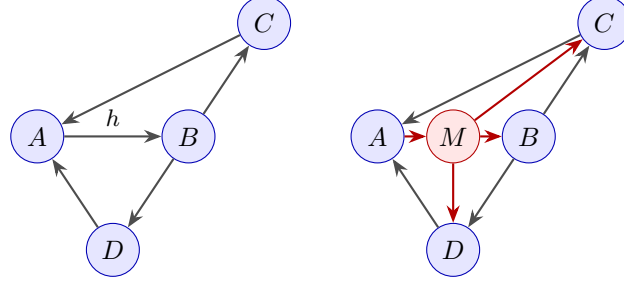
5.1 算法描述

设 $h = he$ (若 $h.face = None$ 但 $twin.face = Some$, 自动改为操作 `twin`), $A = twin.vertex$, $B = h.vertex$ 。 $F_1 = h.face = (A \rightarrow B \rightarrow C \rightarrow A)$, 其中 $n_1 = h.next$ ($B \rightarrow C$), $p_1 = h.prev$ ($C \rightarrow A$)。

目标：在 A 、 B 之间插入中点 M ，将 1 条边分裂为 2 条。

5.1.1 内部边（两侧均有面）

$F_2 = \text{twin.face} = (B \rightarrow A \rightarrow D \rightarrow B)$, $n_2 = \text{twin.next} (A \rightarrow D)$, $p_2 = \text{twin.prev} (D \rightarrow B)$ 。



分裂前：2 面，1 边 $A-B$ 分裂后：4 面，4 边 ($A-M$, $M-B$, $M-C$, $M-D$)

操作步骤：

1. 创建中点 M ，位置 $= \frac{A+B}{2}$ ；
2. 新建 8 条半边： $a \rightarrow m, m \rightarrow a, m \rightarrow b, b \rightarrow m, m \rightarrow c, c \rightarrow m, m \rightarrow d, d \rightarrow m$ ；
3. F_1 重用为 $(A \rightarrow M \rightarrow C \rightarrow A)$ ，新建 $F_{new1} = (M \rightarrow B \rightarrow C \rightarrow M)$ ；
4. F_2 重用为 $(B \rightarrow M \rightarrow D \rightarrow B)$ ，新建 $F_{new2} = (M \rightarrow A \rightarrow D \rightarrow M)$ ；
5. 复用 n_1, p_1, n_2, p_2 ，仅修改其 **next/prev/face**；
6. 删除 h , twin 。

5.1.2 边界边（仅一侧有面）

$\text{twin.face} = \text{None}$ 。新建 6 条半边（无 $m \rightarrow d, d \rightarrow m$ ）， $\text{twin} (B \rightarrow A \text{ 边界})$ 分裂为 $b \rightarrow m$ ($B \rightarrow M$ 边界) + $m \rightarrow a$ ($M \rightarrow A$ 边界)，不创建 F_{new2} 。同时维护边界环的 **next/prev** 链：记 twin 的前后邻居为 p, n ，则重链为

$$p \rightarrow (B \rightarrow M) \rightarrow (M \rightarrow A) \rightarrow n,$$

并把 $p.\text{next}$ 、 $n.\text{prev}$ 改指向新边界半边，否则边界环断裂、破坏拓扑。

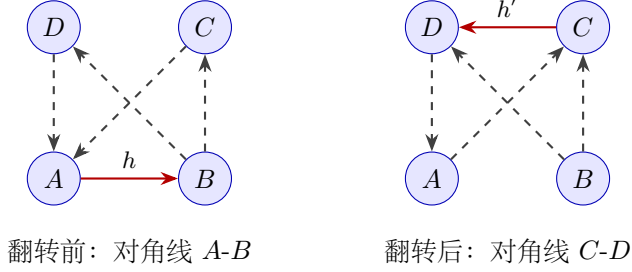
5.2 数量变化

情形	ΔV	ΔF	ΔE_{he} (半边数)
内部边	+1	+2	+6 (删 2 加 8)
边界边	+1	+1	+4 (删 2 加 6)

6 flip_edge: 内部边翻转

6.1 算法描述

设 $h = \text{he}$, $A = \text{twin.vertex}$, $B = h.\text{vertex}$ 。 $F_1 = h.\text{face} = (A \rightarrow B \rightarrow C \rightarrow A)$, $F_2 = \text{twin.face} = (B \rightarrow A \rightarrow D \rightarrow B)$ 。 四边形 $A-B-C-D$ 的对角线 $A-B$ 翻转为 $C-D$ 。



翻转后 h 从 $A \rightarrow B$ 变为 $D \rightarrow C$ ($\text{vertex} = C$), twin 从 $B \rightarrow A$ 变为 $C \rightarrow D$ ($\text{vertex} = D$)。

$$F'_1 = (D \rightarrow C \rightarrow A \rightarrow D) = h \rightarrow p_1 \rightarrow n_2 \rightarrow h \quad (1)$$

$$F'_2 = (C \rightarrow D \rightarrow B \rightarrow C) = \text{twin} \rightarrow p_2 \rightarrow n_1 \rightarrow \text{twin} \quad (2)$$

6.2 重连规则

半边	原 next / prev	新 next	新 prev
h	n_1 / p_1	p_1	n_2
twin	n_2 / p_2	p_2	n_1
p_1	h / n_1	n_2	h
n_2	p_2 / twin	h	p_1
p_2	twin / n_2	n_1	twin
n_1	p_1 / h	twin	p_2

面归属变更: $n_2.\text{face}$ 从 F_2 改为 F_1 , $n_1.\text{face}$ 从 F_1 改为 F_2 。

6.3 校验

- $h.\text{face}$ 与 twin.face 均须为 Some (内部边);
- $C \neq D$ (防退化);
- C 与 D 不相连 (否则翻转会产生重复边/非流形, 返回 `FlipCreatesNonManifoldEdge`)。

数量不变: $\Delta V = \Delta F = \Delta E_{\text{he}} = 0$ 。

7 collapse_edge: 边折叠

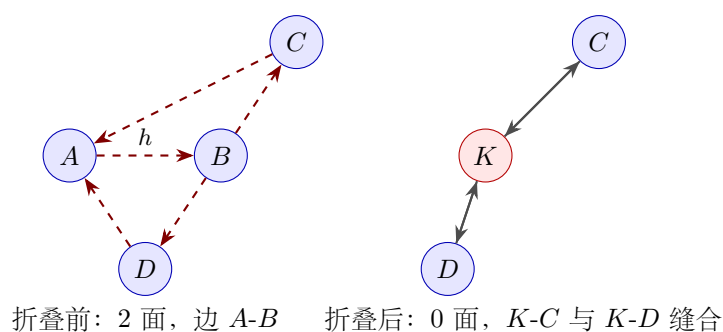
7.1 算法描述

设 $h = \text{he}$, $A = \text{twin.vertex}$, $B = h.\text{vertex}$ 。 $F_1 = h.\text{face} = (A \rightarrow B \rightarrow C \rightarrow A)$, $F_2 = \text{twin.face} = (B \rightarrow A \rightarrow D \rightarrow B)$ 。合并 A 、 B 为新顶点 K , 删除 F_1 、 F_2 。

新顶点位置 `collapse_edge` 固定使用 A 、 B 中点作为 K 的位置:

$$\mathbf{p}_K = \frac{\mathbf{p}_A + \mathbf{p}_B}{2}.$$

`collapse_edge_at(mesh, he, pos)` 则使用调用者指定的 $\mathbf{p}_K = \text{pos}$, 用于 QEM 减面等需要最优折叠位置的场景。两者共享同一核心实现 `collapse_edge_impl`, 仅 `target_pos` 参数不同 (None $\rightarrow$ 中点), 拓扑修改逻辑完全一致。



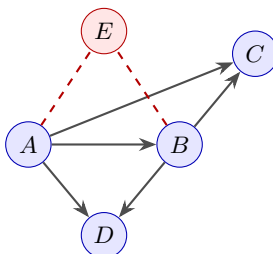
7.2 链接条件

定理: 在流形三角网格中折叠边 $A-B$ 不产生非流形, 当且仅当

$$\text{link}(A) \cap \text{link}(B) = \text{link}(AB) = \{C, D\}$$

其中 $\text{link}(V)$ 是 V 的邻居集合, $\text{link}(AB)$ 是 A 、 B 的公共邻居。

直觉: 若 A 、 B 有额外公共邻居 $E \notin \{C, D\}$, 则折叠后 $K-E$ 边会被两个不同面共享 (来自原 $A-E$ 和 $B-E$), 产生非流形。



$$\begin{aligned} \text{公共邻居} &= \{C, D, E\} \supsetneq \{C, D\} \\ &\Rightarrow \text{链接条件违反} \end{aligned}$$

边界顶点限制 即使链接条件满足, A 、 B 同时为边界顶点时折叠仍被拒绝。合并两个边界顶点会让同一顶点在边界环中出现两次, 破坏边界拓扑 (产生非流形边界顶点)。

7.3 操作步骤

1. 校验: $h.face, twin.face$ 均 Some; $C \neq D$; 链接条件满足; A, B 不同时为边界顶点。
2. 收集: 遍历 A, B 的 outgoing 环, 收集所有 $vertex = A$ 或 B 的存活半边 (排除待删除的 $h, twin, n_1, p_1, n_2, p_2$)。
3. 创建 K (位置 $= \frac{A+B}{2}$)。
4. 缝合 twin:

$$p_1.twin.twin \leftarrow n_1.twin \quad (3)$$

$$n_1.twin.twin \leftarrow p_1.twin \quad (4)$$

$$n_2.twin.twin \leftarrow p_2.twin \quad (5)$$

$$p_2.twin.twin \leftarrow n_2.twin \quad (6)$$

5. 批量更新: 所有收集到的半边的 $vertex \leftarrow K$ 。
6. 更新 outgoing: $K.halfedge \leftarrow p_1.twin$ (现 $K \rightarrow C$); 若 C 或 D 的 outgoing 指向已删除半边, 更新为 $n_1.twin$ 或 $n_2.twin$ 。
7. 删除: $h, twin, n_1, p_1, n_2, p_2, F_1, F_2, A, B$ 。

7.4 数量变化

$$\Delta V = -1, \quad \Delta F = -2, \quad \Delta E_{he} = -6$$

8 extrude_face: 面挤出

8.1 算法描述

设三角面 $F = (v_0, v_1, v_2)$ CCW, 半边 $h_0(v_0 \rightarrow v_1), h_1(v_1 \rightarrow v_2), h_2(v_2 \rightarrow v_0)$, 对应边界 twin $t_0(v_1 \rightarrow v_0), t_1(v_2 \rightarrow v_1), t_2(v_0 \rightarrow v_2)$ (均 $face = None$)。

目标: 沿 $\vec{o}$ (offset) 方向把 F 挤出为闭合三棱柱——底面 F 保留, 顶面 $F' = (v'_2, v'_1, v'_0)$ 反向 CCW (外法向向外), 三条侧边各分为 2 个三角形。

8.1.1 限制条件

F 的三条边必须均为边界边 ($twin.face = None$), 否则返回 **Inconsistent**。这保证挤出不会破坏邻接面的拓扑。

8.1.2 退化检查

- $\|\vec{o}\| < 10^{-12}$: 返回 **DegenerateTriangle**;
- 任一侧面三角形面积 $< 10^{-12}$: 返回 **DegenerateTriangle**。

侧面三角形 $T_{1,i} = (v_{i+1}, v_i, v'_i)$ 的面积为

$$\text{area}_i = \frac{1}{2} |\vec{e}_i \times \vec{o}|, \quad \vec{e}_i = \text{pos}_{i+1} - \text{pos}_i$$

当 $\vec{o}$ 与 $\vec{e}_i$ 平行时侧面退化。

8.1.3 创建元素

- 3 个新顶点 v'_0, v'_1, v'_2 , 位置 = $\text{pos}_i + \vec{o}$;
- 7 个新面: 1 顶面 F' + 6 侧面三角形 (每条侧边 2 个);
- 18 条新半边, 重用 3 条原边界 twin t_0, t_1, t_2 作为侧面半边。

8.1.4 顶面 F'

$F' = (v'_2, v'_1, v'_0)$ CCW (反向), 半边

$$\text{tp}_0(v'_2 \rightarrow v'_1), \quad \text{tp}_1(v'_1 \rightarrow v'_0), \quad \text{tp}_2(v'_0 \rightarrow v'_2)$$

形成 $\text{tp}_0 \rightarrow \text{tp}_1 \rightarrow \text{tp}_2 \rightarrow \text{tp}_0$ 闭合环。

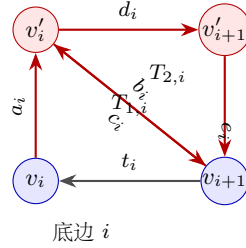
8.1.5 侧面三角形

对每条侧边 $i \in \{0, 1, 2\}$ (indices mod 3):

$$T_{1,i} = (v_{i+1}, v_i, v'_i) = t_i \rightarrow a_i \rightarrow b_i \rightarrow t_i$$

$$T_{2,i} = (v_{i+1}, v'_i, v'_{i+1}) = c_i \rightarrow d_i \rightarrow e_i \rightarrow c_i$$

其中 $a_i(v_i \rightarrow v'_i)$ 、 $b_i(v'_i \rightarrow v_{i+1})$ 、 $c_i(v_{i+1} \rightarrow v'_i)$ 、 $d_i(v'_i \rightarrow v'_{i+1})$ 、 $e_i(v'_{i+1} \rightarrow v_{i+1})$ 均为新半边。



8.1.6 twin 对应 (12 对)

半边对	关系	说明
$t_i \leftrightarrow h_i$	已存在	底面边 (保留原 twin)
$a_i \leftrightarrow e_{i-1}$	垂直边	相邻侧面共享 $v_i \rightarrow v'_i$
$b_i \leftrightarrow c_i$	对角线	侧面内对角线
$d_i \leftrightarrow \text{tp}_{\text{map}(i)}$	顶面边	顶面与侧面共享

其中顶面 twin 映射 map:

$$d_0 \leftrightarrow \text{tp}_1, \quad d_1 \leftrightarrow \text{tp}_0, \quad d_2 \leftrightarrow \text{tp}_2$$

(因 d_0 为 $v'_0 \rightarrow v'_1$, 与 tp_1 ($v'_1 \rightarrow v'_0$) 互为 twin; 其他类推。)

8.1.7 顶点 outgoing 入口

新顶点 v'_i 的 outgoing 入口设为 d_i (origin = v'_i , 因 d_i 的 twin 是 $tp_{\text{map}(i)}$, 其 vertex = v'_i)。
原顶点 v_i 的 outgoing 不变 (仍指向某条原半边, 绕 v_i 的 CCW 环已通过 a_i/e_{i-1} 扩展)。

8.2 校验

操作完成后调用 `validate_topology` (来自 `validate.rs`) 执行完整校验: twin 互指、next/prev 一致、面边界环长度 = 3、入口字段一致、退化面、流形约束。若存在错误, 回滚并返回 `Inconsistent`。

8.3 数量变化

$$\Delta V = +3, \quad \Delta F = +7, \quad \Delta E_{\text{he}} = +18$$

挤出后 Euler 示性数 $\chi = V - E + F$:

$$\underbrace{3 - 3 + 1}_{\text{原三角 (盘)} = 1} \xrightarrow{\text{extrude}} \underbrace{6 - 12 + 8}_{\text{三棱柱 (球)} = 2}$$

即从带边界的盘变为闭合三棱柱 (同胚于球面, $\chi = 2$)。

8.4 批量版本

`extrude_faces(mesh, &[FaceId], offset)` 逐个调用 `extrude_face`, 累积返回所有新建的面 ID。若两面共享边, 先挤出的面会使共享边变为内部边, 后续挤出会因「面含内部边」返回 `Inconsistent`——此时网格已部分修改, 调用方需自行处理回滚或仅在互不相连的面集合上使用批量接口。

9 extrude_region: 区域挤出

`extrude_region(mesh, &[FaceId], offset)` 模仿 Blender/Maya 中“Extrude Region”的行为: 选中区域内所有面共同移动 `offset`, 区域内部边不产生侧面, 仅在区域边界生成侧面四边形 (三角化)。

9.1 顶点分类

对每个区域顶点 v , 用 `VertexAdjacentFaces` 收集其邻接面集合 $\mathcal{A}(v)$:

$$\text{内部顶点} \iff \mathcal{A}(v) \neq \emptyset \wedge \mathcal{A}(v) \subseteq \mathcal{R}$$

其中 $\mathcal{R}$ 为区域面集合。内部顶点原地平移 $\text{pos} += \vec{o}$, $\text{vert_map}[v] = v$; 边界顶点创建新顶点 $v' = \text{pos} + \vec{o}$, $\text{vert_map}[v] = v'$ 。

9.2 半边分类

对每条区域半边 h ，检查 $h.\text{twin.face}$ ：

$$\text{内部半边} \iff h.\text{twin.face} \in \mathcal{R}$$

- **内部半边**：直接更新 $h.\text{vertex} \leftarrow \text{vert_map}[v_t]$ 。twin 同理（也在区域内，会被独立更新）。无需新半边。
- **边界半边**：创建新顶半边 h_{top} ($\text{vertex} = \text{vert_map}[v_t]$) 替换 h 在面环中的位置； h 被释放供侧面使用。

9.3 侧面创建

每条边界半边 $h(v_o \rightarrow v_t)$ 生成四边形 (v_o, v_t, v'_t, v'_o) ，沿对角线 $v_o \rightarrow v'_t$ 三角化：

$$\begin{aligned} T_1 &= (v_o, v_t, v'_t) : h \rightarrow \text{up}_t \rightarrow \text{diag}_{\text{rev}} \\ T_2 &= (v_o, v'_t, v'_o) : \text{diag} \rightarrow s_{\text{top}} \rightarrow \text{down}_o \end{aligned}$$

其中：

- $\text{up}_t : v_t \rightarrow v'_t$ ， $\text{down}_o : v'_o \rightarrow v_o$ ：垂直半边，每个边界顶点创建一对 (up, down)，相邻侧面共享。
- $s_{\text{top}} : v'_t \rightarrow v'_o$ ：顶边，是 h_{top} 的 twin。
- $\text{diag} : v_o \rightarrow v'_t$ ， $\text{diag}_{\text{rev}} : v'_t \rightarrow v_o$ ：对角线对。

9.4 面环重建

每个区域面 F 的半边环中，边界半边 h 替换为 h_{top} ，内部半边保留。新环 $r_0 \rightarrow r_1 \rightarrow r_2 \rightarrow r_0$ 的朝向与原面一致（不反转），因此 F 复用为顶面，朝向不变。

9.5 退化检查

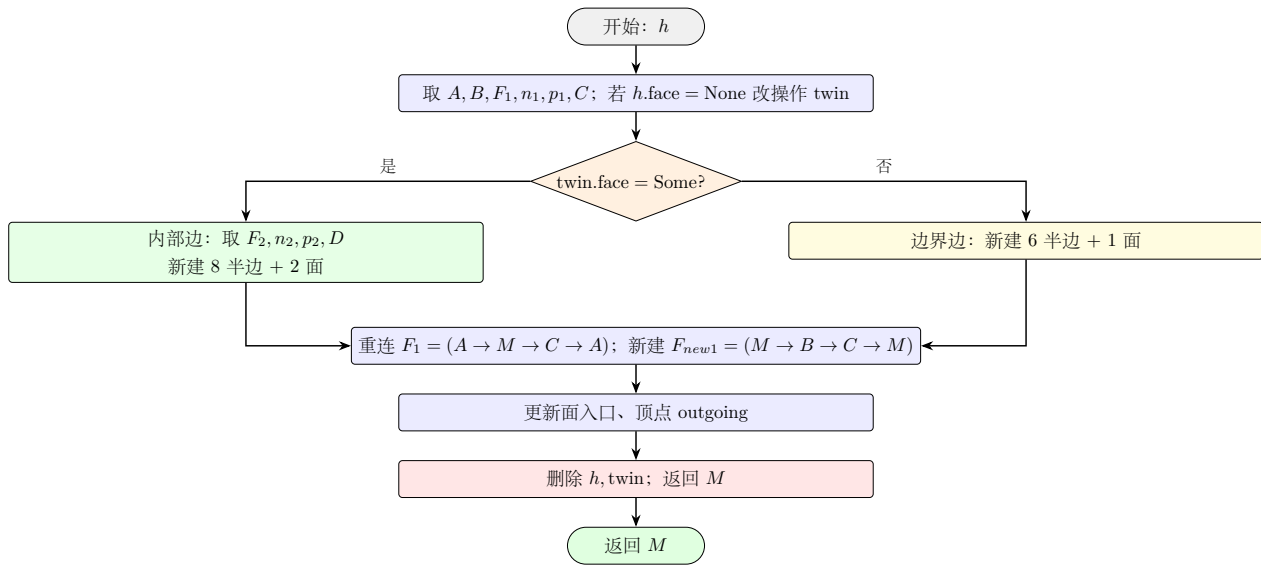
- $\|\vec{o}\| < 10^{-12}$ 返回 `DegenerateTriangle`
- 任一侧面面积 $\frac{1}{2}|\vec{e}_i \times \vec{o}| < 10^{-12}$ 返回 `DegenerateTriangle`

9.6 边界情况

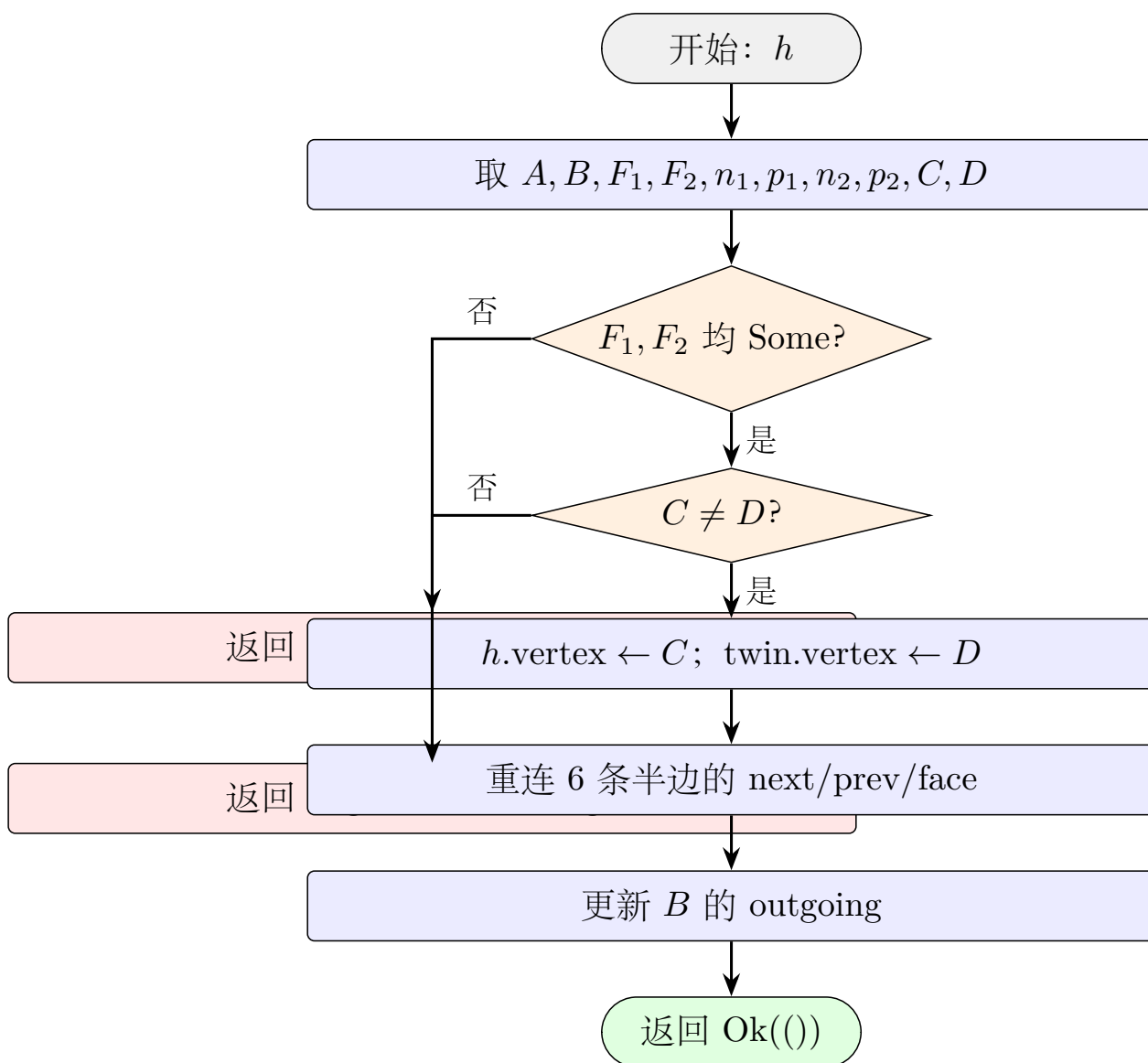
- **区域包含整个网格**：所有顶点为内部顶点，仅平移，无侧面。
- **区域非连通**：各连通分量独立挤出，共享同一 offset。
- **空区域**：返回 `Inconsistent`。

10 算法流程图

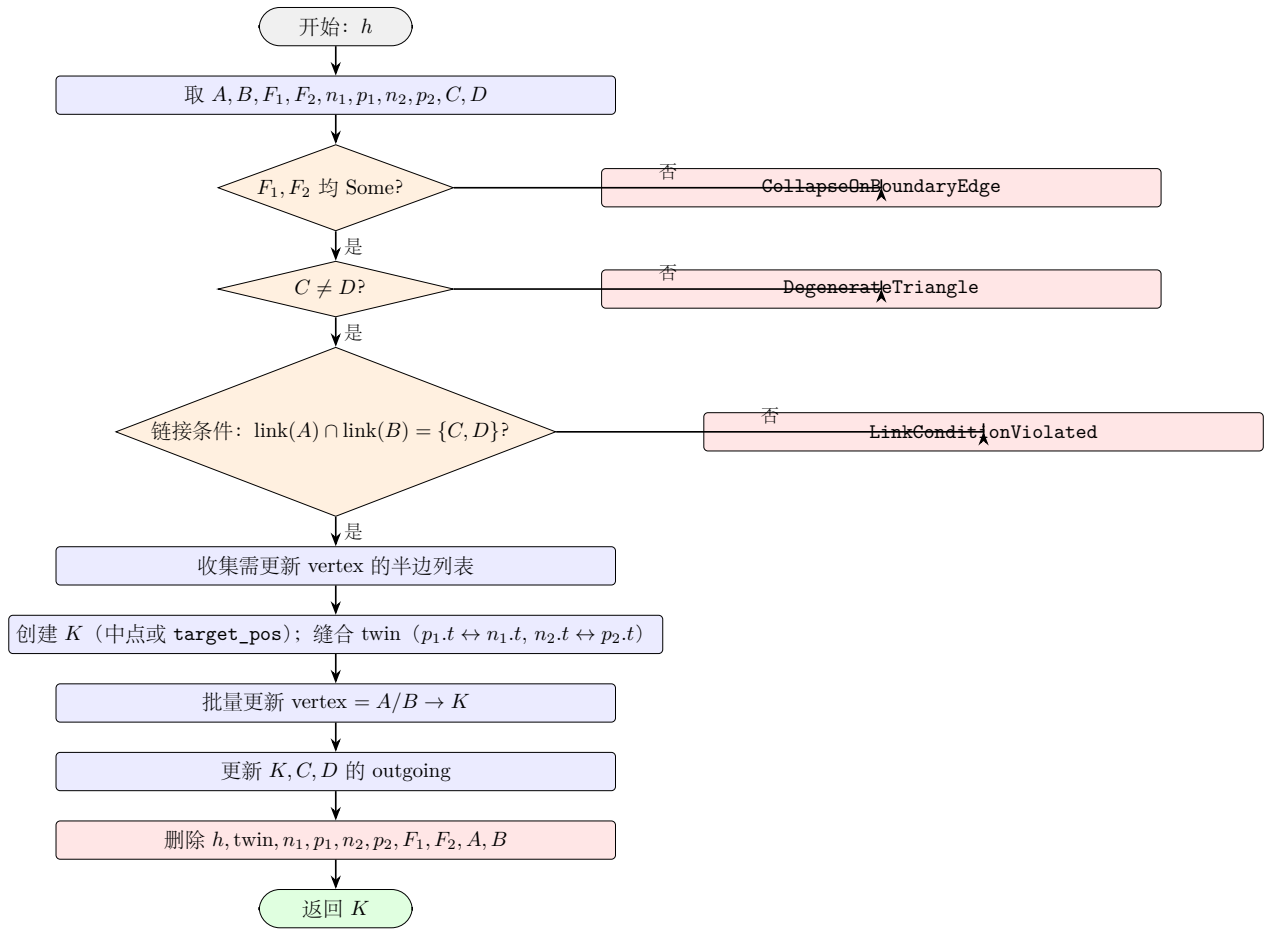
10.1 split_edge 流程



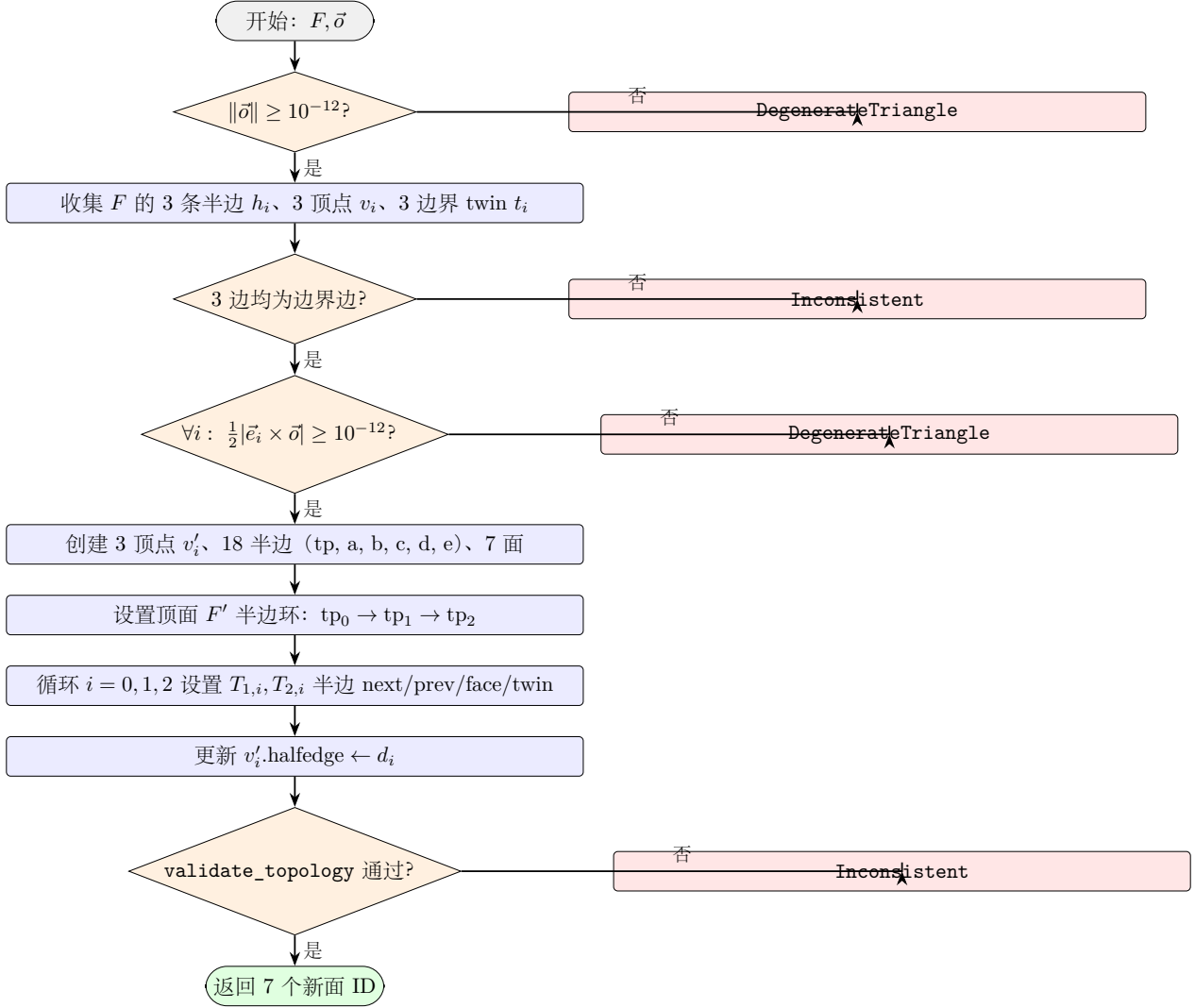
10.2 flip_edge 流程



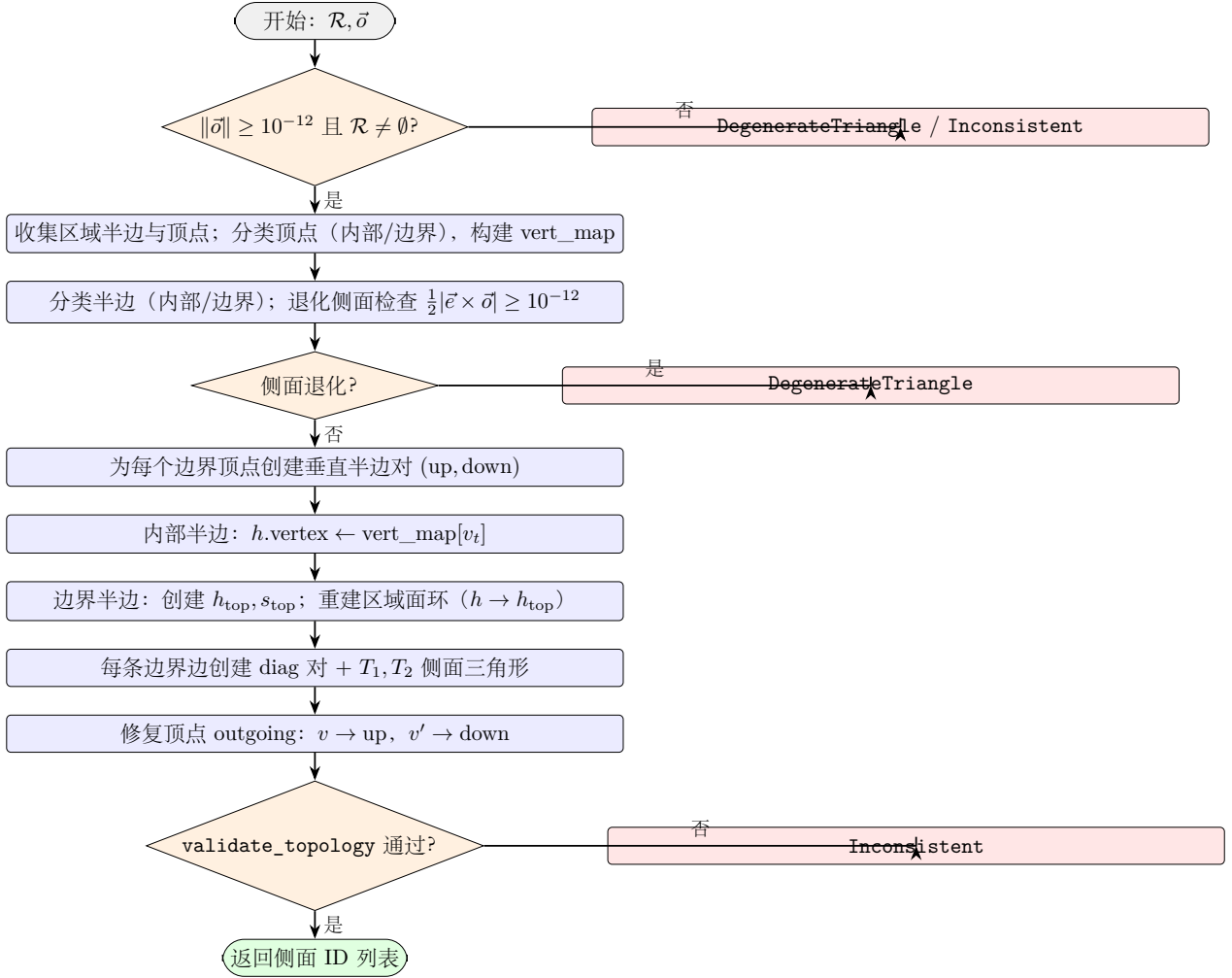
10.3 collapse_edge 流程



10.4 extrude_face 流程



10.5 extrude_region 流程



11 add_triangle: 增量三角形构建

`add_triangle(mesh, v0, v1, v2)` 向网格追加一个 CCW 三角形 ($v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_0$), 自动处理 twin 配对与顶点 outgoing 入口。它是不依赖 `build_mesh_from_vertices_and_faces` 的细粒度构建 API, 适合流式增量构造 (如从索引流重建网格、布尔运算结果回写)。

11.1 算法步骤

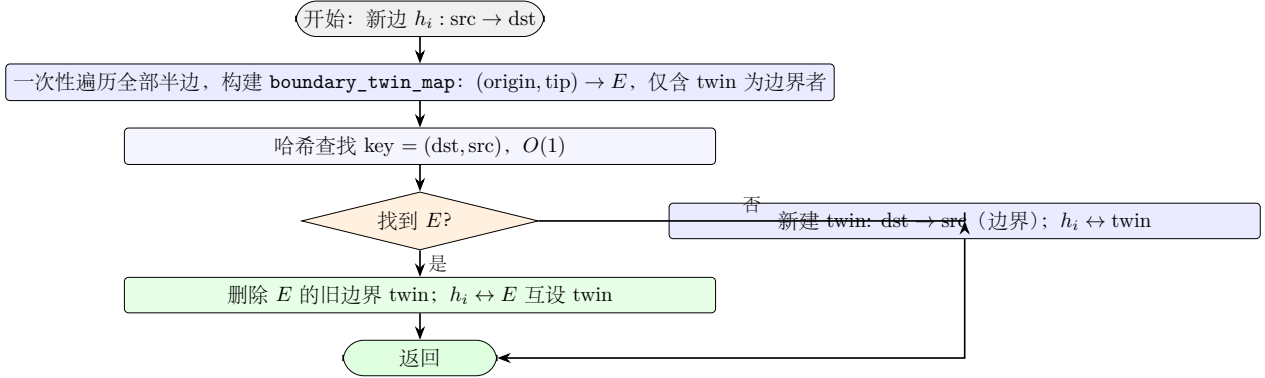
1. **退化校验:** $v_0 \neq v_1 \neq v_2 \neq v_0$, 且三个顶点均存在。
2. **创建 3 条半边:** $h_0 : v_0 \rightarrow v_1$, $h_1 : v_1 \rightarrow v_2$, $h_2 : v_2 \rightarrow v_0$, 设置 next/prev 闭环 $h_0 \rightarrow h_1 \rightarrow h_2 \rightarrow h_0$ 。
3. **创建面 F ,** $\text{halfedge} \leftarrow h_0$, 三条半边 $\text{face} \leftarrow F$ 。
4. **twin 配对:** 首先一次性遍历所有现存半边, 构建哈希索引 $\text{boundary_twin_map} : (\text{origin}, \text{tip}) \rightarrow \text{HalfEdgeId}$, 仅收录 twin 为边界半边 ($\text{face} = \text{None}$) 的内部半边。然后对每条新边 $h_i : \text{src} \rightarrow \text{dst}$, 以 $\text{key} = (\text{dst}, \text{src})$ 在哈希中 $O(1)$ 查找满足条件的 E (E 从 $\text{dst} \rightarrow \text{src}$, 其 twin 从 $\text{src} \rightarrow \text{dst}$ 且为边界半边)。

- **命中**：删除旧边界 twin，将 h_i 与 E 互设为 twin（复用已有内部半边）。
- **未命中**：新建边界半边 $\text{twin} : \text{dst} \rightarrow \text{src}$ ，与 h_i 互指。

5. **outgoing 入口**：对 v_0, v_1, v_2 ，若其 **halfedge** 字段为 **None**，设置为对应的 h_i 。

6. **最终校验**：调用 `validate_mesh`。

11.2 twin 查找策略



11.3 数量变化

每次调用： $\Delta V = 0$ 、 $\Delta F = +1$ 。半边数变化取决于共享边数量：

共享边数 k	ΔE_{he}	说明
0（孤立三角形）	+6	3 内部 + 3 边界 twin
1（与现有网格共享一边）	+4	2 新内部 + 2 新边界 twin（旧 twin 被删除复用）
2（嵌入两面之间）	+2	1 新内部 + 1 新边界 twin
3（封闭体最后一面）	+0	全部 twin 已存在

11.4 校验

- 三顶点两两不同（`DegenerateTriangle`）；
- 三顶点均存在（`Inconsistent`）；
- `validate_mesh` 通过。

12 split_face：面分裂（poke）

`split_face(mesh, face)` 在三角面重心处插入新顶点 M ，将 1 个三角形分裂为 3 个三角形（又称 *poke* 或 *face center insertion*）。常用于均匀加密网格、生成径向辐条拓扑。

12.1 算法步骤

设原面 $F = (v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow v_0)$ ，对应半边 h_0, h_1, h_2 ，其 twin 分别为 t_0, t_1, t_2 （部分可能为 None）。

1. 校验： F 存在且为三角面（边界环长度 = 3）。

2. 计算重心：

$$\mathbf{p}_M = \frac{\mathbf{p}_{v_0} + \mathbf{p}_{v_1} + \mathbf{p}_{v_2}}{3}.$$

3. 创建中心顶点 M ，位置 = $\mathbf{p}_M$ 。

4. 删除旧面与 3 条半边 h_0, h_1, h_2 （保留旧 twin t_0, t_1, t_2 ）。

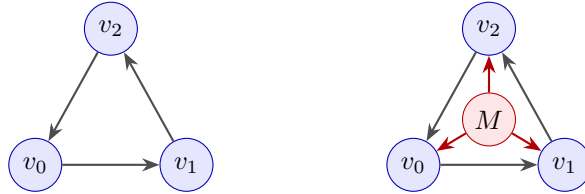
5. 创建 3 个新面，对 $i \in \{0, 1, 2\}$ ：

- 新建辐条半边 $\text{spoke_out}_i : M \rightarrow v_i$ 与 $\text{spoke_in}_i : v_{i+1} \rightarrow M$ ，互为 twin；
- 新建外边界半边 $\text{outer}_i : v_i \rightarrow v_{i+1}$ ，恢复与 t_{i+1} 的 twin 关系；
- 设置 next/prev 闭环： $\text{spoke_out}_i \rightarrow \text{outer}_i \rightarrow \text{spoke_in}_i \rightarrow \text{spoke_out}_i$ ；
- 新建面 F_i ， $\text{halfedge} \leftarrow \text{spoke_out}_i$ 。

6. 设置 M 的 outgoing： $\leftarrow \text{spoke_out}_0$ 。

7. 校验：`validate_mesh`。

12.2 拓扑示意



分裂前：1 面

分裂后：3 面，3 条辐条边

12.3 twin 恢复策略

原外边界半边 $h_i : v_i \rightarrow v_{i+1}$ 被删除，其 twin $t_i : v_{i+1} \rightarrow v_i$ 保留。新建的 $\text{outer}_i : v_i \rightarrow v_{i+1}$ 与 t_i 重新互设 twin，从而保持外边界邻接面的拓扑不变。

12.4 数量变化

$$\Delta V = +1, \quad \Delta F = +2, \quad \Delta E_{\text{he}} = +3$$

（删 3 旧半边，加 3 outer + 6 spoke = 9 新半边；外边界 twin 关系复用。）

12.5 注意事项

`split_face` 当前**无单元测试**。建议补充以下用例：

- 单三角面 poke: 验证 $+1V + 2F + 3HE$ 与重心位置；
- 内部三角面 poke: 验证邻接面 `twin` 关系不被破坏；
- 多次连续 poke: 所有面仍为三角面；
- 非三角面 poke: 返回 `Inconsistent`。

13 复杂度

设边的度（邻接面数）为 d_e （三角网格中 $d_e \in \{1, 2\}$ ），顶点的度为 d_v 。

操作	时间复杂度
<code>split_edge</code>	$O(1)$ （固定数量的半边创建与重连）
<code>flip_edge</code>	$O(1)$ （重连 6 条半边）
<code>collapse_edge</code>	$O(d_A + d_B)$ （遍历 outgoing 环收集 + 更新）
<code>extrude_face</code>	$O(1)$ （固定 18 半边 + 7 面创建）+ $O(E + F)$ （校验）
<code>extrude_faces</code>	$O(n) \cdot (\text{extrude_face} + \text{validate})$
<code>extrude_region</code>	$O(\mathcal{R} \cdot \bar{d}_v) + O(\partial\mathcal{R})$ （顶点分类 + 侧面创建）+ $O(E + F)$ （校验）
<code>add_triangle</code>	$O(E_{\text{he}})$ （一次性构建哈希索引）+ $O(1)$ （3 次查找）+ $O(E + F)$ （校验）
<code>split_face</code>	$O(1)$ （固定 9 半边创建与重连）+ $O(E + F)$ （校验）
<code>validate_mesh</code>	$O(E_{\text{he}} + F)$ （遍历所有半边与面）

14 单元测试覆盖

`src/topology_ops.rs` 末尾共 38 个测试，分八组（`split_face` 暂无测试）：

测试名	覆盖点
校验	
<code>validate_clean_meshes</code>	三种 fixture 均通过校验
<i>split_edge</i>	
<code>split_boundary_edge_of_single_triangle</code>	边界边分裂: $+1V + 1F + 4HE$ ，中点位置正确
<code>split_interior_edge_of_two_triangles</code>	内部边分裂: $+1V + 2F + 6HE$
<code>split_boundary_edge_passed_as_boundary_halfedge</code>	传入边界半边自动转 <code>twin</code>
<code>split_boundary_edge_maintains_boundary_loop</code>	真实 builder 边界边分裂后，边界环 <code>next/prev</code> 完整
<i>flip_edge</i>	
<code>flip_interior_edge_of_two_triangles</code>	内部边翻转: 数量不变, $h.\text{vertex} = C$
<code>flip_boundary_edge_fails</code>	边界边翻转返回错误

<code>flip_then_flip_restores_topology</code>	双翻恢复拓扑
<code>flip_preserves_boundary_vertices</code>	翻转后四角仍为边界顶点
<code>flip_edge_rejects_when_opposite_vertices_connected</code>	四面体中 (c, d) 已相连, 翻转被拒绝
<i>collapse_edge</i>	
<code>collapse_interior_edge_of_closed_fan</code>	闭合扇形折叠: $-1V - 2F - 6HE$
<code>collapse_boundary_edge_fails</code>	边界边折叠返回错误
<code>collapse_with_violated_link_condition_fails</code>	5 顶点 4 面网格, 公共邻居 3 个
<code>collapse_with_both_boundary_vertices_fails</code>	对角线两端点均为边界顶点, 拒绝折叠
<code>collapse_interior_edge_with_one_boundary_vertex</code>	折叠「一个边界 + 一个内部」, 边界环保持完整
<code>collapse_link_condition_check_function</code>	直接测试 <code>check_link_condition</code>
<code>collapse_then_remaining_mesh_valid</code>	折叠后 K 为边界顶点, 网格合法
<code>collapse_edge_uses_midpoint_position</code>	<code>collapse_edge</code> 新顶点 = $\frac{A+B}{2}$
<code>collapse_edge_at_uses_custom_position</code>	<code>collapse_edge_at</code> 新顶点 = 指定位置
<code>collapse_edge_at_preserves_topology_counts</code>	两函数拓扑计数一致, 仅位置不同
<code>collapse_edge_at_on_boundary_edge_fails</code>	<code>collapse_edge_at</code> 同样禁止边界边
<i>extrude_face</i>	
<code>extrude_single_triangle</code>	单三角挤出: $+3V + 7F + 18HE$, Euler = 2
<code>extrude_zero_offset_returns_error</code>	零向量 <code>offset</code> 返回 <code>DegenerateTriangle</code>
<code>extrude_degenerate_side_face_returns_error</code>	<code>offset</code> 平行边时侧面退化
<code>extrude_face_with_internal_edge_returns_error</code>	面含内部边返回 <code>Inconsistent</code>
<code>extrude_faces_batch_disjoint</code>	3 个不相交三角形批量挤出
<code>extrude_face_from_built_mesh</code>	<code>build_mesh_from_vertices_and_faces</code> 后挤出
<code>extrude_face_invalid_face_id</code>	无效 <code>FaceId</code> 返回 <code>Inconsistent</code>
<code>extrude_face_preserves_boundary_after_extrude</code>	挤出后无边界半边 (闭合体)
<i>extrude_region</i>	
<code>extrude_region_single_face_of_tetrahedron</code>	闭合四面体单面挤出: $+3V + 6F + 18HE$, Euler = 2
<code>extrude_region_two_adjacent_faces</code>	双相邻面挤出: 共享边为内部边不产生侧面, $+4V + 8F + 24HE$
<code>extrude_region_three_faces_leaving_cap</code>	三面挤出: 1 内部顶点原地平移, 剩余面成盖子
<code>extrude_region_zero_offset_returns_error</code>	零 <code>offset</code> 返回 <code>DegenerateTriangle</code>
<code>extrude_region_empty_faces_returns_error</code>	空区域返回 <code>Inconsistent</code>
<code>extrude_region_degenerate_side_returns_error</code>	<code>offset</code> 平行边界边时侧面退化
综合与错误处理	
<code>split_then_validate_all_faces_triangular</code>	连续 <code>split</code> 后所有面仍为三角
<code>invalid_halfedge_returns_error</code>	无效句柄三个操作均返回 <code>InvalidHalfEdge</code>
<i>add_triangle</i>	
<code>add_triangle_to_empty_mesh</code>	空网格上添加首三角形, 6 半边 / 3 顶点齐全
<code>add_two_adjacent_triangles</code>	第二三角形与第一共享边, 自动 <code>twin</code> 缝合
<code>add_degenerate_triangle_fails</code>	重复顶点返回 <code>DegenerateTriangle</code>
<code>add_triangle_with_invalid_vertex_fails</code>	无效 <code>VertexId</code> 返回 <code>InvalidVertex</code>
<code>add_triangle_preserves_existing_edge</code>	共享边已存在时复用半边, 拓扑保持

全库 cargo test 当前共 371 个用例, 全部通过。分布: topology_ops (38)、traversal (62)、subdiv (41)、geometry (41)、storage (20)、query (19)、property (16)、decimate (15)、bvh (15)、io (22)、validate (9)、builtin_attrs (9)、test_util (8)、connectivity (8)、primitives (7)、boolean (7)、triangulation (6)、remesh (5)、export (5)、linalg (4)、

conformal (4)、weld (3)、orientation (2)、ids (2)、geodesics (2)、parameterization (1)。

15 设计取舍

- 为何 `split_edge` 自动处理边界半边？调用方可能传入边界半边 (`face = None`)，操作语义应与传入其 `twin` 一致。内部统一转为「`h.face = Some`」再处理，避免重复代码。
- 为何 `flip_edge` 禁止边界边？翻转边界边会改变网格边界拓扑，且四边形只有一侧有面时翻转无意义。
- 为何 `collapse_edge` 用链接条件而非简单度检查？链接条件是流形边折叠的充要条件，度检查只能捕获部分非流形情形。例如 A 、 B 度均为 4 但公共邻居只有 2 个时折叠仍然合法。
- 为何操作前先 `clone` 数据？Rust 借用检查要求修改 `mesh` 时不能同时持有 `&HalfEdge` 引用。先 `clone` 所需字段到局部变量，再进行修改，是最简洁的解法。
- 为何 `collapse_edge` 中先缝合 `twin` 再更新 `vertex`？若先更新 `vertex`，`to_update` 列表中的半边可能因 `vertex` 变化而难以判断原值。先缝合 `twin` 确保结构完整，再批量更新 `vertex`。
- 为何 `collapse_edge_at` 与 `collapse_edge` 共享 `_impl`？QEM 减面需要指定最优折叠位置 p_K （由二次误差矩阵求得），但拓扑修改逻辑与中点折叠完全一致。提取 `collapse_edge_impl` 避免代码重复，`collapse_edge` 和 `collapse_edge_at` 各仅一行委托，符合「始终保持代码整洁」原则。
- 为何 `extrude_face` 限制面三边均为边界边？若面含内部边，挤出后该内部边的邻接面会被孤立（共享边变成新顶面/侧面的边），破坏流形性。此限制保证挤出操作只在「自由面」上进行，类似 OpenMesh 的 `triangulate` 限制。
- 为何顶面 F' 反向 CCW？底面 $F = (v_0, v_1, v_2)$ 法向 $\vec{n}_F$ 朝外，顶面 F' 应法向 $-\vec{n}_F$ 朝外（向外即向上），故 CCW 序列取反： (v'_2, v'_1, v'_0) 。这保证挤出后整个闭合体的法向一致朝外。
- 为何每条侧边用 2 个三角形而非 1 个四边形？本库仅支持三角网格，侧面四边形必须沿对角线三角化：

$$T_{1,i} = (v_{i+1}, v_i, v'_i), \quad T_{2,i} = (v_{i+1}, v'_i, v'_{i+1})$$

- 为何 `extrude_region` 复用原面为顶面而非新建？区域挤出中，原区域面 F 的法向已朝外（假设输入网格法向一致），挤出后 F 移动到顶面位置，朝向应保持不变。复用 F 避免了删除原面 + 新建顶面的开销，且内部边的 `twin` 关系天然保留（仅更新 `vertex` 字段）。这与 `extrude_face` 的策略（保留底面 + 新建反向顶面）不同，因为 `extrude_face` 处理的是孤立面，而 `extrude_region` 处理的是嵌入在网格中的面，需要保持与非区域面的拓扑连接。
- 为何 `extrude_region` 内部顶点原地平移而非复制？内部顶点的所有邻接面均在区域内，全部会随挤出移动。复制顶点会导致原顶点变成孤立点（无邻接面），破坏流形性。原地平移 $v.pos += \vec{o}$ 是唯一正确的选择。

- 为何 `extrude_region` 边界顶点的 `outgoing` 指向 `up`? 边界顶点 v 既连接非区域面（保留原位），又连接侧面（新创建）。 v 的 `outgoing` 入口必须指向一个确实从 v 出发的半边。垂直半边 $up: v \rightarrow v'$ 是最稳定的选择，因为它不依赖于具体的边界边排列顺序。
- 为何 `add_triangle` 使用临时哈希而非持久化邻接表? `add_triangle` 每次调用时一次性遍历全部半边构建 `boundary_twin_map` ($O(|E_{he}|)$)，然后 3 次查找 $O(1)$ 。相比旧版每条边独立扫描 ($O(3|E_{he}|)$) 减少约 3 倍常数。持久化邻接表会增加 `MeshStorage` 内存开销并破坏与 `slotmap` 的简洁集成；大规模重建仍应使用 `build_mesh_from_polygons`（一次成型）。
- 为何 `split_face` 删除原半边再重建，而非就地分裂? `poke` 操作将 3 条外边界半边全部「易主」给新的 3 个面，并新增 6 条辐条半边。若就地修改，需要为每条原半边分配新 `twin`、调整 `next/prev`，分支较多。先删除再重建使逻辑清晰、对称，且 `slotmap` 的 `key` 复用策略保证内存不会无限增长。